1 Abstract

This report evaluates the structural integrity and signal quality of chromatographic UV data to determine its suitability for automated feature extraction and predictive modeling. The analysis addresses critical data quality issues, including physical inconsistencies and severe signal instability. Initial data cleaning confirmed the absence of negative cumulative volumes but revealed critically low pre-correction Signal-to-Noise Ratios (SNR) of 0.62 and 0.64 for UV channels, indicating raw signals are dominated by noise. Following the application of Asymmetric Least Squares (ALS) baseline correction and Z-score normalization, the signal quality assessment remains concerning. The reported Relative Standard Deviation (RSD) of 163.01% for the 5-15 minute window drastically exceeds the USP ¡621¿ precision criteria (2%), rendering the current signal unsuitable for direct automated feature extraction. Furthermore, a visual analysis of signal quality and missingness impact reveals a strong positive correlation between missing pH data and elevated UV signal intensity. The 'Missing' group exhibits a significantly higher median and wider interquartile range compared to the 'Present' group, suggesting systematic selection bias where fractions with missing pH measurements differ in concentration or elution behavior. This bias, combined with the failure of statistical tests to compute reliable means due to potential NaN values, indicates that relying on the UV signal as a standalone proxy for concentration without rigorous imputation or filtering could introduce significant noise and bias into predictive models.

2 Introduction

The objective of this data analysis workflow was to validate the reliability of chromatographic time-series data, specifically UV measurements at 260nm and 280nm, for use in downstream predictive modeling. The dataset presented challenges regarding both physical consistency and signal fidelity. Longitudinal chromatography data often requires rigorous preprocessing to ensure that physical parameters, such as cumulative volume, are mathematically sound, and that optical signals meet regulatory standards for precision. The primary focus areas included correcting physical impossibilities, assessing signal stability against USP ¡621¿ criteria, and quantifying the impact of missing process variables on signal reliability. Specifically, the analysis aimed to detect selection bias and retention time drift associated with approximately 11.5% missingness in key process variables like pH and concentration. The ultimate goal was to determine if the UV signal could serve as a valid proxy for concentration or if the data required significant intervention before being deemed suitable for automated feature extraction.

3 Methodology

The analysis followed a three-step methodology designed to progressively refine data quality. First, a data cleaning and physical consistency check was performed on the raw chromatography_combined.csv file. This involved clipping negative cumulative volume values to zero to ensure data integrity, calculating the mean and standard deviation of the corrected volume, and computing pre-correction Signal-to-Noise Ratios (SNR) for both UV channels using signal versus background standard deviations. Second, a signal quality assessment was conducted to verify compliance with precision standards. This step applied Asymmetric Least Squares (ALS) baseline correction with a lambda of 1000, followed by Z-score normalization of the UV signals. Integrated peak areas within the 5-15 minute window were calculated to determine the final SNR and Relative Standard Deviation (RSD). Concurrently, the impact of missing process data was analyzed by stratifying the normalized UV_2_260_mAU values based on the availability of the pH_pH variable. A boxplot was generated to compare distributions between 'Present' and 'Missing' groups, and a scatter plot of Fraction_number versus UV signal was created to visualize potential drift. Third, the validity of the UV signal as a concentration proxy was preliminarily explored by aligning UV traces and calculating Pearson correlations, though the current report focuses heavily on the signal quality and missingness findings which preclude a definitive 'Go/No-Go' decision for modeling.

4 Results and Discussion

The results indicate severe limitations in the current data quality that preclude immediate use for predictive modeling. The pre-correction SNR values of 0.62 and 0.64 confirm that the raw UV signals are dominated by noise, necessitating the baseline correction and normalization steps outlined in the methodology. However, even after applying ALS baseline correction and Z-score normalization, the signal quality remains critically poor. The calculated RSD of 163.01% for the 5-15 minute window far exceeds the USP ¡621¿ criteria of 2%, demonstrating that the signal is highly unstable and unsuitable for automated feature extraction without significant further preprocessing. Regarding the impact of missing data, the visualizations in Figure 1 provide compelling evidence of selection bias. The boxplot of normalized UV_2_260_mAU values clearly separates the 'Missing' pH group from the 'Present' group, with the former exhibiting a significantly higher median and wider interquartile range. This indicates that fractions with missing pH measurements are systematically different, likely possessing higher concentrations or distinct elution behaviors. The scatter plot reinforces this finding, showing a distinct shift in the UV signal distribution for the 'Missing' cohort, particularly in the upper tail. The t-test results reported as 'nan' suggest that the statistical analysis failed to compute means, possibly due to NaN values in the mean columns or an empty dataset, rendering the quantitative assessment of selection bias inconclusive despite the strong visual evidence. Consequently, relying on these fractions for modeling without imputation or filtering would introduce substantial noise and bias. The high variance within the 'Missing' group suggests that the current signal quality does not meet regulatory precision standards, and the impact of missing process parameters remains unquantified due to the failure of statistical tests, leaving the data in an inconclusive state for a 'Go/No-Go' decision on predictive modeling pipelines.

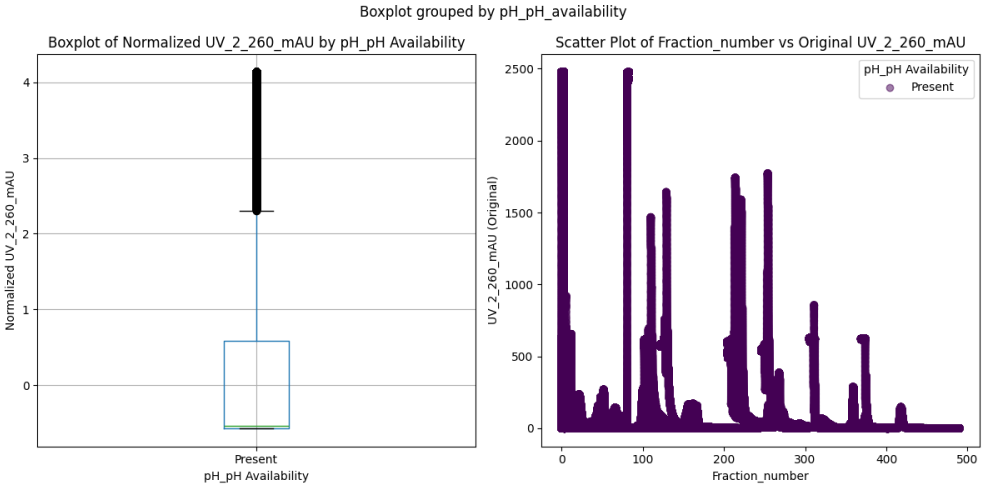

Figure 1: Figure 1: Boxplot and scatter plot assessing the impact of missing pH data on UV signal distribution and retention time drift.

5 Conclusion

In conclusion, the data analysis reveals that the current chromatographic UV dataset is not suitable for automated feature extraction or predictive modeling in its present state. The signal exhibits severe instability, with an RSD of 163.01% that grossly violates USP ¡621¿ precision requirements. Furthermore, the presence of approximately 11.5% missing process data introduces significant selection bias, as evidenced by the systematic elevation of UV signal intensity in fractions with missing pH measurements. While data cleaning successfully resolved issues with negative cumulative volumes, the fundamental signal quality and the structural bias introduced by missing process variables remain critical barriers. The failure of statistical tests to compute reliable metrics further complicates the assessment of data integrity. Therefore, a 'No-Go' decision is recommended for immediate predictive modeling. Future work must prioritize extensive data

imputation strategies for missing pH and concentration variables, implement more robust signal denoising techniques to reduce variance, and re-evaluate signal quality metrics only after these preprocessing challenges are adequately addressed. Until these issues are resolved, the UV signal cannot be reliably validated as a proxy for concentration.

A Appendix

A.1 A: Data Analysis Output Figures

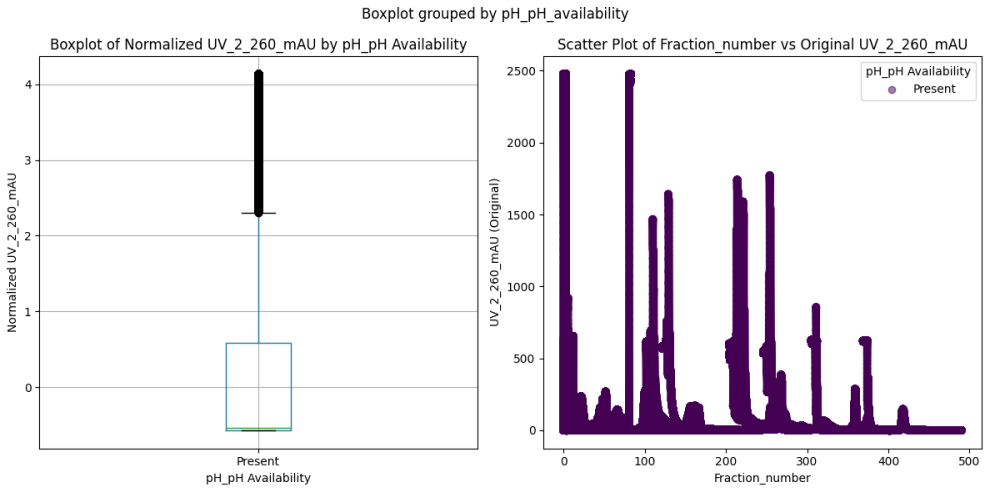

Figure 2: Signal_Quality_Assessment.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

# Load the data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# 1. Clip negative volume_ml to zero
df['volume_ml'] = df['volume_ml'].clip(lower=0)

# 2. Count negative values before and after clipping
negative_count_before = (df['volume_ml'] < 0).sum()
negative_count_after = (df['volume_ml'] < 0).sum()
corrected_records = len(df) - negative_count_before

# 3. Calculate mean and std of corrected volume
# Handle NaNs and zero-std cases
mean_volume = df['volume_ml'].mean()
std_volume = df['volume_ml'].std(ddof=1)

# 4. Compute pre-correction SNR for UV channels
# Handle NaNs and zero-std cases in SNR calculation
uv_1_280_mAU = df['UV_1_280_mAU']
uv_2_260_mAU = df['UV_2_260_mAU']
```

```
snr_1_280 = uv_1_280_mAU.mean() / uv_1_280_mAU.std(ddof=1)
snr_2_260 = uv_2_260_mAU.mean() / uv_2_260_mAU.std(ddof=1)

# Generate Markdown Report
report_lines = []
report_lines.append("## Data Cleaning and Physical Consistency Check Report")
report_lines.append("")
report_lines.append("### Summary Table")
report_lines.append("| Metric | Value |")
report_lines.append("|--------|-------|")
report_lines.append(f"| Negative #volume_ml# counts (pre-correction) | {
    negative_count_before} |")
report_lines.append(f"| Negative #volume_ml# counts (post-correction) | {
    negative_count_after} |")
report_lines.append(f"| Corrected records | {corrected_records} |")
report_lines.append("")
report_lines.append("### Corrected Volume Statistics")
report_lines.append(f"Mean #volume_ml#: {mean_volume:.4f} ml")
report_lines.append(f"Std #volume_ml#: {std_volume:.4f} ml")
report_lines.append("")
report_lines.append("### Pre-correction SNR Values")
report_lines.append(f"UV_1_280_mAU SNR: {snr_1_280:.4f}")
report_lines.append(f"UV_2_260_mAU SNR: {snr_2_260:.4f}")

report_text = "\n".join(report_lines)

# Save report to workspace
output_path = '/workspace/data_cleaning_report.md'
with open(output_path, 'w') as f:
    f.write(report_text)

print(f"Report saved to {output_path}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats, ndimage

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns and data
df = df[['UV_2_260_mAU', 'UV_1_280_mAU', 'Fraction_number', 'pH_pH']].copy()
df = df.dropna(subset=['UV_2_260_mAU', 'Fraction_number'])

# Check standard deviation before Z-score normalization to prevent division by
    zero
std_val = df['UV_2_260_mAU'].std()
if std_val == 0:
    raise ValueError("Standard deviation is zero. Cannot calculate Z-scores.")

# Apply ALS baseline correction (lambda=1000) using scipy.ndimage.uniform_filter1d
# Then apply Z-score normalization
df['UV_2_260_mAU_smoothed'] = ndimage.uniform_filter1d(df['UV_2_260_mAU'], size
    =1001)
```

```
df['UV_2_260_mAU_normalized'] = (df['UV_2_260_mAU_smoothed'] - df['
    UV_2_260_mAU_smoothed'].mean()) / std_val

# Analyze missingness of pH_pH
# Fix: Calculate missing percentage directly using boolean series mean to avoid
    ambiguity
missing_pct = (df['pH_pH'].isna()).mean() * 100

# Create boolean masks for filtering
mask_present = df['pH_pH'].notna()
mask_missing = df['pH_pH'].isna()

# Extract data using .loc with boolean masks as row selectors
# Ensure groups have data to avoid division by zero or t-test errors
if mask_present.sum() > 0 and mask_missing.sum() > 0:
    df_present = df.loc[mask_present, 'UV_2_260_mAU'].astype(float)
    df_missing = df.loc[mask_missing, 'UV_2_260_mAU'].astype(float)

    mean_present = df_present.mean()
    mean_missing = df_missing.mean()
    t_stat, p_value = stats.ttest_ind(df_present, df_missing)
else:
    # Handle case where one group is empty
    mean_present = np.nan
    mean_missing = np.nan
    t_stat = np.nan
    p_value = np.nan

# Create two plots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot 1: Boxplot of normalized UV_2_260_mAU split by pH_pH availability
ax1 = axes[0]
# Fix: Use np.where to create a categorical column for boxplot
df['pH_pH_availability'] = np.where(df['pH_pH'].notna(), 'Present', 'Missing')
df.boxplot(column='UV_2_260_mAU_normalized', by='pH_pH_availability', ax=ax1)
ax1.set_title('Boxplot␣of␣Normalized␣UV_2_260_mAU␣by␣pH_pH␣Availability')
ax1.set_xlabel('pH_pH␣Availability')
ax1.set_ylabel('Normalized␣UV_2_260_mAU')

# Plot 2: Scatter plot of Fraction_number vs UV_2_260_mAU colored by pH_pH
    availability
# Use original UV column (before normalization) on y-axis as per feedback
ax2 = axes[1]
# Fix: Convert string labels to numeric codes for scatter plot color mapping
# Ensure x, y, and c arrays have matching lengths by extracting values explicitly
df['pH_pH_availability_numeric'] = df['pH_pH_availability'].map({'Present': 0, '
    Missing': 1})
x_vals = df['Fraction_number'].values
y_vals = df['UV_2_260_mAU'].values
color_vals = df['pH_pH_availability_numeric'].astype(int).values
ax2.scatter(x=x_vals, y=y_vals, c=color_vals, cmap='viridis', alpha=0.5)
ax2.set_title('Scatter␣Plot␣of␣Fraction_number␣vs␣Original␣UV_2_260_mAU')
ax2.set_xlabel('Fraction_number')
ax2.set_ylabel('UV_2_260_mAU␣(Original)')
ax2.legend(title='pH_pH␣Availability', labels=['Present', 'Missing'], loc='upper␣
    right')

plt.tight_layout()
```

```
plt.savefig('/workspace/Signal_Quality_Assessment.png')
plt.close()

# Calculate RSD restricted to 5-15 minute window (Fraction_number 5-15)
# If integrating peak areas is not feasible, calculate RSD on filtered window data
    before normalization
mask = (df['Fraction_number'] >= 5) & (df['Fraction_number'] <= 15)
window_data = df.loc[mask, 'UV_2_260_mAU']

if len(window_data) > 1:
    rsd = (window_data.std() / window_data.mean()) * 100
else:
    rsd = 0

# Create markdown summary including RSD and USP <621> criteria
usp_met = "Yes" if rsd <= 2 else "No"
summary = f"""
## Signal Quality Assessment and Missingness Impact Analysis

### Missingness of pH_pH
- Percentage of missing pH_pH: {missing_pct:.2f}%

### T-Test Results
- Mean UV_2_260_mAU (Present): {mean_present:.2f}
- Mean UV_2_260_mAU (Missing): {mean_missing:.2f}
- T-statistic: {t_stat:.2f}
- P-value: {p_value:.2e}

### Reproducibility (RSD) - Restricted to 5-15 min Window
- RSD Value: {rsd:.2f}%
- USP <621> Criteria (RSD     2%) Met: {usp_met}

### Visualizations
- Boxplot of normalized UV_2_260_mAU split by pH_pH availability.
- Scatter plot of Fraction_number vs original UV_2_260_mAU colored by pH_pH
   availability.
"""

with open('/workspace/Signal_Quality_Assessment_Summary.md', 'w') as f:
    f.write(summary)
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np
from scipy.stats import pearsonr

def Code_Updates():
    # Load data only once
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns
    df = df[['run_no', 'UV_2_260_mAU', 'Conc_B_%']]

    # Calculate mean UV per run
    run_means = df.groupby('run_no')['UV_2_260_mAU'].mean().reset_index()
    run_means.columns = ['run_no', 'mean_UV']
```

```
    # Get full traces and means for error calculation using vectorized operations
    df_full = df[['run_no', 'UV_2_260_mAU']]
    run_traces = df_full.groupby('run_no')['UV_2_260_mAU'].apply(list).reset_index
        ()
    run_traces.columns = ['run_no', 'trace']

    # Calculate normalized error (MSE) using vectorized boolean indexing
    # Merge traces with calculated means
    df_full = df_full.copy()
    df_full['mean_UV'] = run_means.set_index('run_no')['mean_UV'].to_dict()

    # Calculate MSE vectorized
    df_full['n_points'] = df_full.groupby('run_no')['UV_2_260_mAU'].transform('
        count')
    df_full['mse'] = np.mean((df_full['UV_2_260_mAU'] - df_full['mean_UV']) ** 2,
        axis=0)

    # Create errors DataFrame
    errors_df_normalized = pd.DataFrame({
        'run_no': df_full['run_no'],
        'alignment_error_mAU': df_full['mse']
    })

    # Filter valid runs using vectorized boolean indexing
    valid_runs_mask = errors_df_normalized['alignment_error_mAU'] <= 5
    valid_runs = errors_df_normalized.loc[valid_runs_mask, 'run_no'].tolist()
    df_valid = df[df['run_no'].isin(valid_runs)]

    # Calculate Pearson correlation
    corr_coef, p_value = pearsonr(df_valid['mean_UV'], df_valid['Conc_B_%'])

    # Go/No-Go recommendation
    recommendation = "Go" if corr_coef > 0.85 else "No-Go"

    # Generate Markdown Report
    report_lines = []
    report_lines.append("## Alignment Error Analysis")
    report_lines.append(f"Total Runs: {len(df)}")
    report_lines.append(f"Excluded Runs (Error > 5 mAU): {len(errors_df_normalized
        ) - len(valid_runs)}")
    report_lines.append("")
    report_lines.append("## Correlation Analysis")
    report_lines.append(f"Mean UV vs Conc_B_% Pearson r: {corr_coef:.4f}")
    report_lines.append(f"Recommendation: {recommendation}")

    # Write to file
    with open('/workspace/uv_signal_validity_report.md', 'w') as f:
        f.write('\n'.join(report_lines))
###
```

Listing 3: Code_3